Design Patterns in FinTrack

Design Patterns Assignment: one Structural, one Behavioural and one Creational pattern

1. Introduction

The scenario chosen for this assignment is FinTrack, a web application for managing shared expenses within small groups, such as flatmates, friends travelling together or colleagues. Members record the expenses they pay for, the application divides each expense among the people involved, and it shows how much each member owes the others. This allows a group to settle its debts without maintaining a spreadsheet. All amounts are recorded in Indian rupees (INR).

1.1 Features

- **Accounts and authentication.** Users register with an email address and password, or with a Google account. A forgotten password can be reset through a link sent by email.
- **Groups.** Users create groups and invite other people to join them through an email invitation.
- **Expenses.** Any member can record an expense, state who paid and who shared it, and the application divides the amount exactly, to the last paisa.
- **Recurring expenses.** Regular bills, such as rent, can be scheduled to repeat automatically every week or every month.
- **Balances and settlements.** The application calculates who owes whom, reduces the number of payments required, and records a settlement once the receiving member confirms it.
- **Insights.** The application summarises spending by category and over time.
- **Activity and notifications.** Each member receives a feed of the changes made in their groups.
- **Mobile access.** A mobile application uses the same server through its API.

1.2 Technology

FinTrack is written in TypeScript using the Next.js framework. Data is stored in MongoDB, or in the server's memory when no database has been configured. TypeScript programs frequently use plain functions and modules (files) in place of classes. In the class diagrams that follow, each module or function therefore takes the role that a class would take in the standard description of the pattern.

1.3 Patterns identified

One structural, one behavioural and one creational pattern were identified in the FinTrack source code. Each pattern is described under nine headings: intent, justification, motivation, structure, participants, applicability, consequences, collaboration and implementation.

Type	Pattern	Use in FinTrack
Structural	Decorator	Authentication and error handling applied to every API route, in `lib/server/route-helpers.ts`
Behavioural	Strategy	Interchangeable MongoDB and in-memory storage, in `lib/server/store-types.ts`
Creational	Singleton	A single shared database connection, in `lib/db/client.ts`

2. Structural Pattern: Decorator

2.1 Intent

The Decorator pattern attaches additional responsibilities to an object by wrapping it, without modifying the object itself. The wrapper presents the same interface as the object it wraps, so the caller cannot distinguish between the two.

2.2 Justification

FinTrack required every API request to pass through the same security and error-handling steps, without those steps being duplicated in each route. Four approaches were considered, and the Decorator pattern was found to be the most suitable.

Approach	Assessment
Repeating the checks in every route	Although straightforward, this would duplicate the same 10 to 15 lines in every handler. A route that omitted a single line could expose data, and no error would alert the developer.
A common base class inherited by all routes	Next.js requires each route to be an exported function rather than a class. Inheritance would also impose one fixed structure on all routes, although some routes read a request body and others read values from the URL.
Performing the checks in the request proxy (`proxy.ts`)	The proxy runs before the route, but it can only accept or reject a request. It cannot supply the authenticated user or the data store to the handler, nor can it handle errors raised later by the handler.
Decorator (selected)	The checks are written once, applied to each handler by wrapping it, and the authenticated user and data store are passed directly to the handler. The wrapped function remains a standard route from the framework's point of view.

The Decorator pattern was selected because it is the only approach that keeps the checks in a single location, conforms to the route structure that Next.js expects, and covers both the beginning and the end of each request.

2.3 Motivation

When a user opens the Expenses page, the browser sends the request `GET /api/expenses`. Before any expenses can be returned, the server must resolve four concerns:

1. **Identity.** The user may be authenticated through a browser cookie or through a token issued to the mobile application.
2. **Session validity.** If the user has reset their password on another device, any session created before the reset must be rejected.

3. **Data access.** The handler requires the data store, which may be either MongoDB or the in-memory store.
4. **Error handling.** A missing group must produce a 404 response, a forbidden action a 403 response, and an unexpected failure a generic 500 response.

These four concerns apply equally to all 16 route files, including those for groups, settlements, notifications and invitations. Implementing them within each handler would make every handler considerably longer, and the line that carries the route's actual purpose, such as `store.getExpenses(userId)`, would be obscured by the surrounding checks.

There is also a security consequence. If a newly added route omitted the session-validity check, a user who had reset their password after a compromised login would remain exposed through that route. With the Decorator pattern, a developer writes only the route's business logic and wraps it in `withAuth`. All four steps are then applied automatically, and in the same order, to every route.

2.4 Structure

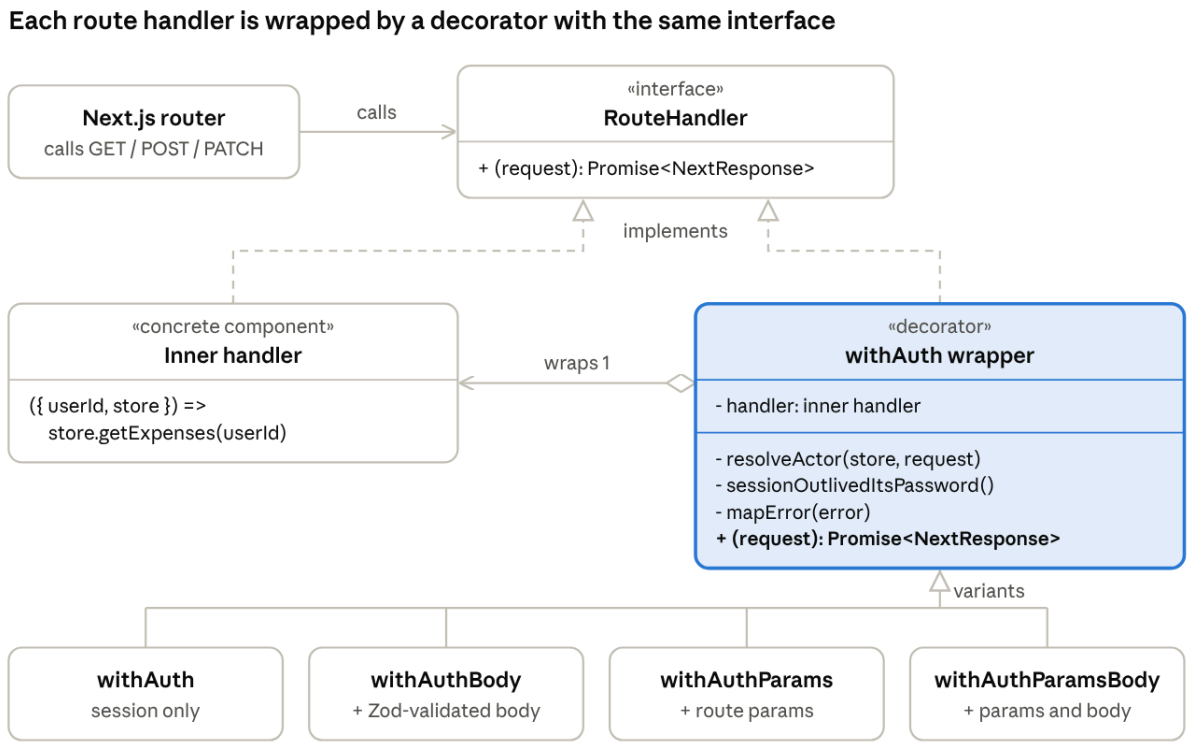


Figure 1. Class diagram of the Decorator in `lib/server/route-helpers.ts`

2.5 Participants

Participant	In FinTrack	Responsibility
Component	The route handler interface, `(request) => Promise<NextResponse>`	Defines the function signature that Next.js invokes for each request.
Concrete Component	The inner handler, for example `store.getExpenses(userId)`	Performs only the work specific to its route.
Decorator	`withAuth` and its variants `withAuthBody`, `withAuthParams` and `withAuthParamsBody`	Wraps the inner handler and applies the shared checks before and after it executes.
Client	The Next.js router	Invokes the wrapped handler in the same manner as an unwrapped one.

2.6 Applicability

The Decorator pattern is applicable when:

- the same additional behaviour must be applied to many objects, as with the checks required on every FinTrack route;
- the behaviour must be added without modifying the objects themselves;
- subclassing would lead to an excessive number of combinations. FinTrack requires four combinations (with or without a request body, and with or without URL parameters), which wrappers accommodate easily.

2.7 Consequences

Benefits

- Each route handler is typically a single line and contains only its own logic.
- A change to the checks is made once and takes effect for every route. Support for mobile authentication was introduced in this way.
- Security errors are less likely, because no route can execute without being wrapped.
- Unexpected errors are logged on the server, while the user receives only a generic message, so internal details are never disclosed to the browser.

Drawbacks

- The processing of a request is divided between the wrapper and the handler, which can make debugging somewhat more difficult.
- A route that requires different checks, such as the public health-check endpoint, must be implemented without the wrapper.

2.8 Collaboration

Next.js invokes the wrapped function exactly as it would invoke an unwrapped handler. The wrapper first obtains the data store and identifies the user, and it rejects the request if the user is not authenticated. Once the checks succeed, it passes the user ID and the data store to the inner handler. When the handler returns, the wrapper sends the result as a JSON response. If the handler raises an error, the wrapper converts it into the appropriate HTTP status code.

2.9 Implementation

- **Functions in place of classes.** Each decorator is a function that accepts a handler and returns a new handler with the same signature. This is the conventional form of the Decorator pattern in TypeScript.
- **Four variants with a shared core.** The variants differ only in whether they also validate the request body or read parameters from the URL. All four use the same authentication function (`resolveActor`) and error-handling function (`mapError`).
- **Input validation.** `withAuthBody` validates the request body against a schema before the handler runs, so the handler never receives invalid data.

```
// app/api/expenses/route.ts
export const GET = withAuth(({ userId, store }) => store.getExpenses(userId));

export const POST = withAuthBody(createExpenseSchema, (body, { userId, store }) =>
  store.createExpense(userId, body)
);
```

3. Behavioural Pattern: Strategy

3.1 Intent

The Strategy pattern defines a family of interchangeable algorithms, places each one behind a common interface, and allows one of them to be selected at runtime. The code that uses the strategy remains independent of the particular algorithm in use.

3.2 Justification

FinTrack required support for two forms of data storage, while every screen, route and business rule had to behave identically with either. Three approaches were considered, and the Strategy pattern was found to be the most suitable.

Approach	Assessment
Supporting MongoDB only	Every developer, tester and demonstration would require a running database. Tests would run more slowly, and the application could not be evaluated without first configuring a database.
An `if (database) … else …` condition in every route	Every storage operation would require two branches within the routes. The branches could easily diverge over time, particularly in the rules that govern which data each user may access.
Strategy (selected)	Both stores implement the same `DataStore` interface. The routes call a single set of methods, and the choice of store is made once, in `getStore()`.

The Strategy pattern was selected because it confines the choice of storage to a single location and allows both stores to be verified by the same set of tests. It also permits a further store to be added later without any change to the routes.

3.3 Motivation

FinTrack operates in two distinct environments:

- **In production,** users' groups, expenses and settlements must be stored permanently, and the application therefore uses **MongoDB**.
- **During development, demonstration and testing,** the application should start immediately with sample data and require no configuration. It therefore uses an **in-memory store**, which is reset whenever the server restarts.

As an example, a user may record a ₹1,200 dinner expense in a group named "Goa Trip". FinTrack must verify that the user belongs to the group, store the expense and record it in the group's activity feed. This sequence must behave identically regardless of which store is in use; otherwise, a defect could arise in production that was never observed during testing.

The access rules make this requirement more significant. A user must never be able to view or modify another group's data. If these rules were implemented in the routes, a newly added route could omit them. The Strategy pattern places the rules within each store instead. Every method receives the ID of the requesting user and verifies it before performing any work, so no route can bypass the check.

3.4 Structure

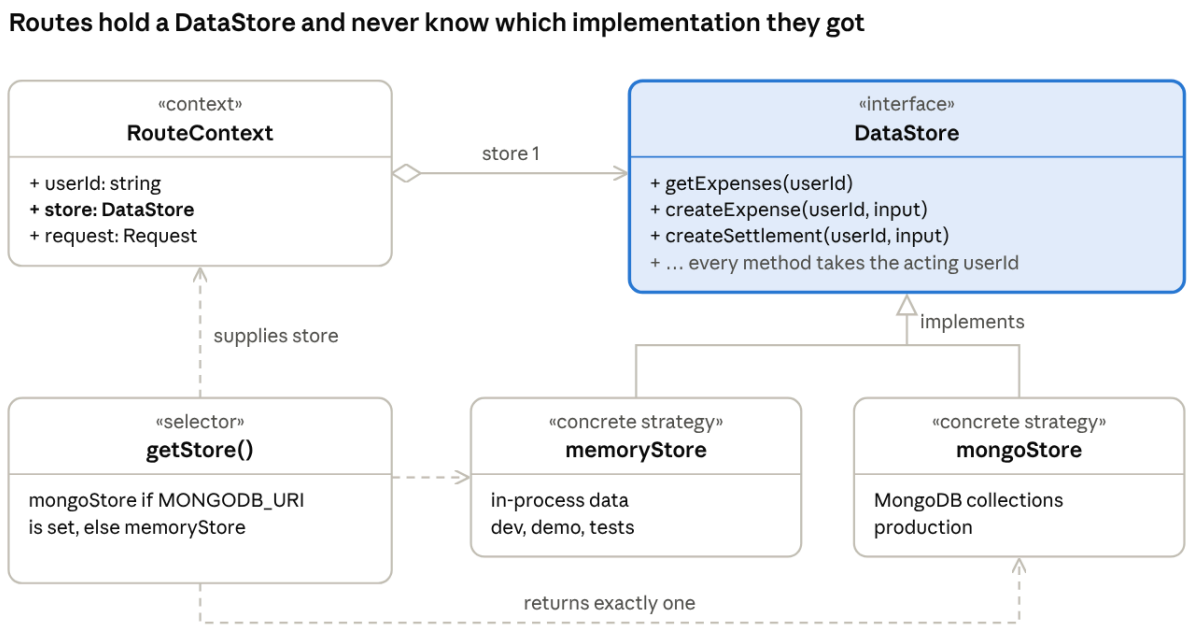


Figure 2. Class diagram of the Strategy in `lib/server/`

3.5 Participants

Participant	In FinTrack	Responsibility
Strategy	The `DataStore` interface	Declares every read and write operation the application requires, such as `getExpenses` and `createSettlement`.
Concrete Strategies	`memoryStore` and `mongoStore`	Each implements every operation, the first using server memory and the second using MongoDB.
Context	`RouteContext` and the route handlers	Hold a reference to a `DataStore` and call its methods without knowing which implementation it is.
Selector	`getStore()`	Selects the store to be used, based on the application's configuration.

3.6 Applicability

The Strategy pattern is applicable when:

- several approaches exist for the same task, such as storing data in memory or in a database;
- the choice of approach must be made at runtime, for example from a configuration setting;

- the code that depends on the task should remain unchanged when a new approach is introduced;
- long chains of conditional statements are to be replaced with separate, independently testable implementations.

3.7 Consequences

Benefits

- Routes and business logic remain unchanged when the storage mechanism changes.
- Developers can run the complete application without installing a database.
- A new store, such as PostgreSQL, could be introduced by writing one additional implementation.
- The same test suite runs against both stores, which prevents them from diverging unnoticed.

Drawbacks

- Every new operation must be implemented twice, once in each store.
- The interface must suit both stores, so features specific to MongoDB cannot be exposed directly.

3.8 Collaboration

For each request, the `withAuth` wrapper calls `getStore()`. If a MongoDB connection string is present in the configuration, the function returns `mongoStore`; otherwise, it returns `memoryStore`. The wrapper places the selected store in the `RouteContext`, and the route handler then calls a method such as `store.createExpense(userId, body)`, which the selected store carries out. Both stores report failures using the same error types, such as "not found" and "forbidden", so the wrapper handles them identically.

3.9 Implementation

- **A common interface.** Both stores are declared with the type `DataStore`. If either one lacks a method, the TypeScript compiler reports an error.
- **Access rules within each store.** Every method receives the ID of the requesting user, and each store verifies that the user is permitted to perform the action.
- **Deferred loading of MongoDB code.** `getStore()` loads the MongoDB store only when a database is configured, which keeps the application smaller in its absence.
- **A shared test suite.** `tests/store-contract.ts` runs the same tests against both implementations.

```
export async function getStore(): Promise<DataStore> {
  if (!features.database) return memoryStore;      // no database configured
  const { mongoStore } = await import("./mongo-store");
  return mongoStore;
}
```

4. Creational Pattern: Singleton

4.1 Intent

The Singleton pattern ensures that a class has only one instance, and provides a single, global point of access to that instance.

4.2 Justification

FinTrack required exactly one database client, shared across the entire server and created only when first needed. Three approaches were considered, and the Singleton pattern was found to be the most suitable.

Approach	Assessment
Creating a new client for every request	Each client opens its own connections. Because each page involves several database reads, the application would quickly reach the database's connection limit, and every request would be delayed while new connections were opened.
Creating the client at server start-up and passing it to every function	Next.js provides no single start-up point at which this could be done, and every function would require an additional parameter. The application would also connect even when no database was configured.
Singleton (selected)	The client is created on the first call to `getMongoClient()` and reused by every subsequent call. Any module can obtain it with a single function call.

The Singleton pattern was selected because it guarantees a single set of connections per server, defers the connection until the database is actually required, and gives all code one consistent means of reaching the database.

4.3 Motivation

A MongoDB client does not open a single connection. It opens a set of connections, known as a **connection pool**, and reuses them across many queries. Establishing this pool is relatively slow, and the database accepts only a limited number of simultaneous connections.

Loading the FinTrack dashboard requires several reads, covering the user, their groups, expenses, settlements and notifications. If each read created its own client, a single page load could open dozens of connections. A small number of users refreshing the page at the same time would exhaust the limit, and the database would begin refusing all further connections.

A second problem arises during development. Whenever a developer saves a file, Next.js reloads the modified code. Without a safeguard, each reload would create a new client while leaving the previous one open, so the number of open connections would grow with every edit.

The Singleton pattern resolves both problems. The first request creates the client, and every subsequent request, from any part of the application, reuses it. In development, the client is additionally stored on `globalThis`, which persists across reloads, so repeated edits continue to use the same client.

4.4 Structure

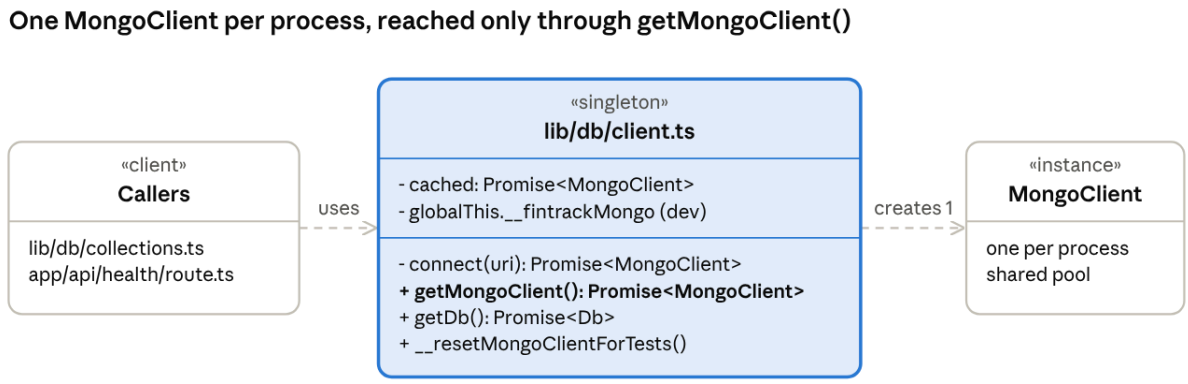


Figure 3. Class diagram of the Singleton in `lib/db/client.ts`

4.5 Participants

Participant	In FinTrack	Responsibility
Singleton	The `lib/db/client.ts` module	Holds the single client in a private variable and exposes `getMongoClient()` as the only means of obtaining it.
Instance	`MongoClient`	The single shared database client.
Clients	`lib/db/collections.ts` and the health-check route	Request the client from the Singleton rather than creating their own.

4.6 Applicability

The Singleton pattern is applicable when:

- exactly one instance of a resource must exist, such as a database client or a connection pool;
- many parts of the program must access that instance through a single, well-known point;
- the instance is costly to create and should be created only once, when it is first required.

4.7 Consequences

Benefits

- The application never opens more than one connection pool per server process.
- The client is created only when first required, so an application without a database never attempts a connection.
- All code accesses the database in the same way, through `getMongoClient()` or `getDb()`.

Drawbacks

- Shared global state makes testing more difficult. FinTrack addresses this with a reset function intended only for tests.
- The single instance conceals a dependency: a function that calls `getDb()` does not declare in its parameters that it requires a database.

4.8 Collaboration

A client calls `getMongoClient()`. On the first call, the module creates a new `MongoClient`, stores it in a private variable named `cached`, and returns it. Every subsequent call returns the stored client, so a second client is never created.

4.9 Implementation

- **Lazy initialisation.** The client is created on the first call rather than when the server starts.
- **No public constructor.** The `connect()` function that creates the client is not exported, so no other module can create a second client.
- **Safety under concurrent requests.** The module stores the *promise* of a client rather than the completed client. Two requests that arrive simultaneously therefore share the same connection attempt.
- **Persistence across development reloads.** In development, the client is also held on `globalThis`, which is not cleared when Next.js reloads a module.
- **Reset for testing.** `__resetMongoClientForTests()` closes and clears the instance so that each test suite begins with a fresh connection.

```
let cached: Promise<MongoClient> | undefined;

export function getMongoClient(): Promise<MongoClient> {
  if (process.env.NODE_ENV === "development") {
    globalThis.__fintrackMongo ??= connect(uri);
    return globalThis.__fintrackMongo;
  }
  cached ??= connect(uri);   // create once, then reuse
  return cached;
}
```

5. Conclusion

Each of the three patterns addresses a concrete problem in FinTrack. The Decorator pattern centralises authentication and error handling rather than repeating them in every route. The Strategy pattern allows the application to alternate between MongoDB and in-memory storage without changes to the remaining code. The Singleton pattern prevents the application from exhausting its database connections. Together, these patterns make the system easier to maintain, more secure and easier to test.